

• Introducere



“Exploding kittens” este un joc de carti strategic pentru oameni pasionati de explozii. Acest joc a fost creat de doi prieteni care, in decursul unei vacante, au isi foloseasca timpul si creativitatea, rezultand un joc de carti.

• Descrierea jocului

Numar jucatori: 2 (jocul real suporta mai multi jucatori)

Jucatorii extrag pe rand carti. Cel care extrage cartea care explodeaza iese din joc daca nu are o carte de “defuse”. Jocul se incheie cand ramane un singur jucator care nu a explodat. Jocul devine atat mai intens pe masura ce se extrag carti din pachetul de pe masa, deoarece mai putine carti sunt in pachet, ceea ce insemna sanse mai mari de a exploda.

Tura unui jucator consta in plasarea pe masa a oricator carti din pachetul din mana. Jucarea continua pana la un jucator care explodeaza, ceea ce insemna scoaterea definitiva din joc a acesteia. Finalul turei este determinat de extragerea unei noi carti din pachetul de pe masa.

Tipuri de carti:

DEFUSE-carte care salveaza jucatorul de la extragerea unei carti de **EXPLODE**. Ulterior folosirii acestora, cartea de **DEFUSE** se poate introduce din nou printre cartile de pe masa. In tot pachetul exista doar **4** carti **DEFUSE**.



EXPLODE-1 singura carte de acest tip este plasata in pachetul de pe masa. Jucatorul care extrage aceasta carte, nu va iese din joc cu conditia ca el sa aiba o carte **DEFUSE**.



ATTACK-forteaza urmatorul jucator sa faca 2 ture, iar jucatorul care a atacat cu aceasta carte isi incheie tura fara sa extraga o carte. In cazul in care un jucator este atacat cu aceasta carte, iar el foloseste o carte tot de acest tip, se va necesita ca jucatorul sa faca 2 ture.



SKIP- carte ce anuleaza o singura tura ce trebuie facuta de jucator (in utilizarea acestei carti duce la terminarea turei si continuarea cu urmasii dar daca jucatorul actual a fost atacat, folosirea cartii nu elimina decat ce trebuie facuta, acesta fiind nevoit sa extraga o carte.



FAVOR- carte ce forteaza un alt jucator sa “doneze” una din cartile lui.
Ex: Jucator1 foloseste FAVOR asupra jucator2, jucator2 alege care carte sa “doneze”, acea carte intra in pachetul din mana jucatorului1.



SHUFFLE- carte ce permite reamestecarea pachetului de carti de pe masa se fac extrageri.



SEE FUTURE- carte ce permite jucatorului ce o foloseste sa vada care urmatoarele 3 carti de pe masa ce pot fi extrase.

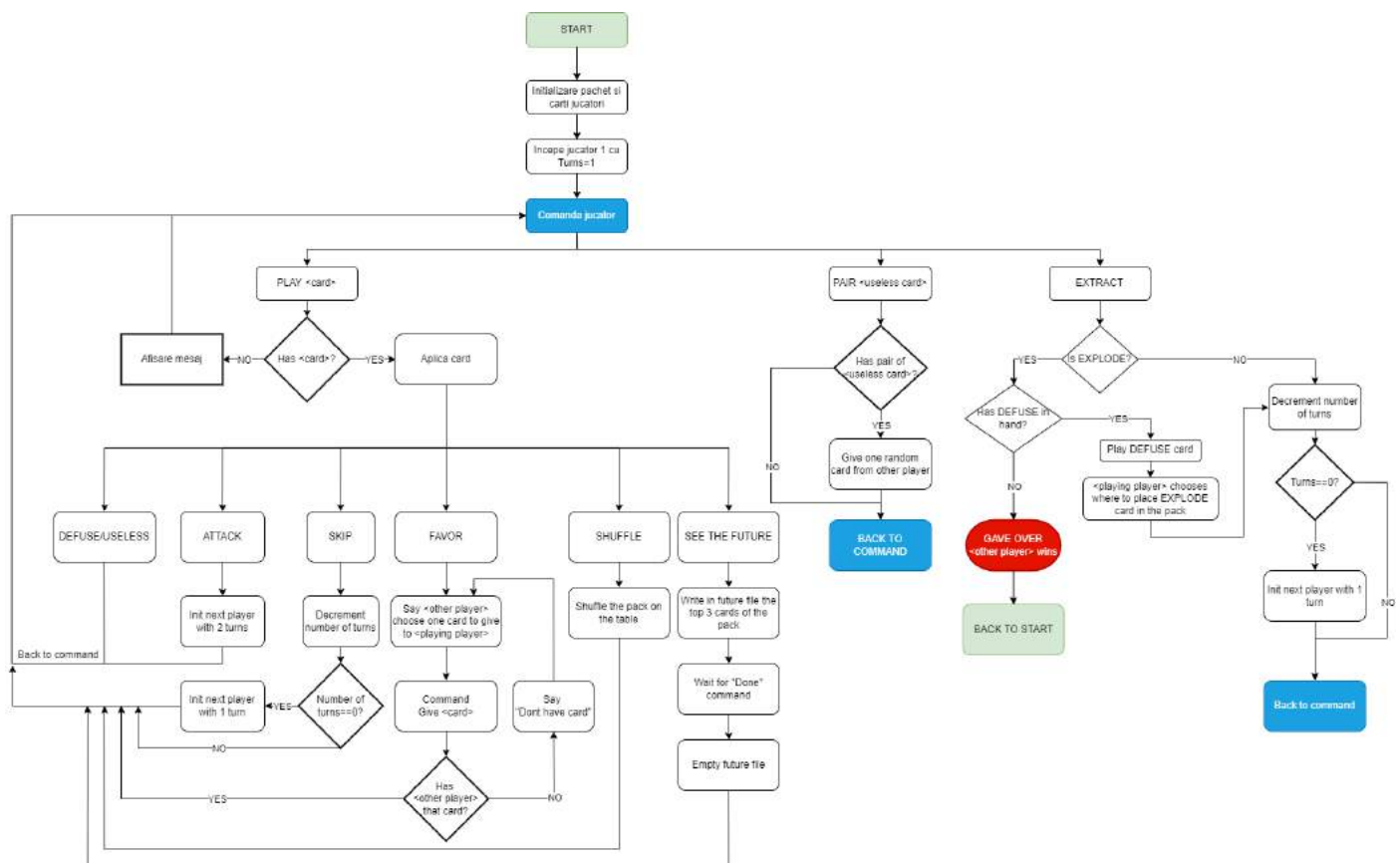


USELESS- carti ce individuale nu au nicio utilitate utilizate singure, dar daca sunt jucate in perechi este alegerea unui jucator si extragerea la intamplare din pachetul acestuia a unei carti.



Astfel, actiunile care le vom implementa pentru acest joc vor fi urmatoarele:

- Start - pentru inceperea jocului, comanda este valida doar la inceputul jocului
- Play <card> - pentru jucarea unei carti
- Pair <useless-card> - pentru jucarea unei perechi de carti useless
- Extract - pentru tragerea unei carti la finalizarea turei
- Give <card> - pentru alegerea carei carti sa fie cedata in cazul utilizarii cartii favor de catre un jucator
- Choose <number> - pentru alegerea locatiei de plasare a cartii EXPLODE dupa utilizarea acesteia
- Done - pentru anuntarea finalizarii de a vizualiza cele 3 din varful stivei de pe masa
- Flow de interactiune



- **Exemplu de executie:**

[] - reprezinta actiuni inafara liniei de comanda

>START

Player 1, look at your cards in file ply1.txt and give a command

[In fisier are cartile:

DEFUSE, SEE_FUTURE, ATTACK,SHUFFLE, TACO_CAT, BEARD_CAT, TACO_CAT, ATTACK]

>PLAY SEE_FUTURE

Player 1, look in future.txt file to see the top 3 cards, write done when you finished

[future.txt contine: 1.EXPLODE, 2.RAINBOW_CAT, 3. FAVOR]

>DONE

Player 1, look at your cards in file ply1.txt and give a command

>PLAY ATTACK

Player 1, your turn is done

Player 2 has two turns to do

Player 2, look at your cards in file ply2.txt and give a command

[ply2.txt: FAVOR, DEFUSE, SKIP, TACO_CAT, BEARD_CAT, BEARD_CAT, SEE_FUTURE, SHUFFLE]

>PLAY SKIP

Player 2 skipped 1 turn

Player 2, look at your cards in file ply2.txt and give a command

>PAIR BEARD_CAT

Player 2 received ATTACK card from player 1

>PLAY FAVOR

Player 1, choose what card to give:

>GIVE SEE_FUTURE

Player 1, you do not have that card

>GIVE TACO_CAT

Player 1 gave away TACO_CAT

Player 2, look at your cards in file ply2.txt and give a command

>PAIR TACO_CAT

Player 2 received DEFUSE card from player 1

>PLAY SHUFFLE

Shuffled the top deck

Player 2, look at your cards in file ply2.txt and give a command

>EXTRACT

Extracted FAVOR card

Player 2, your turn is done

Player 1 has one turn to do

Player 1, look at your cards in file ply1.txt and give a command

>EXTRACT

Extracted EXPLODE card

Player 1 does not have DEFUSE, he dies.

Player 2 wins!

• Implementare

Proiectul este structurat in doua fisiere principale: lex & yacc. In fisierul lex a fost scrisa detectia comenzi specifici jocului (START, PLAY, PAIR ... EXPLODE, SKIP) iar fisierul yacc consta atat in pa cat si in implementarea masinii de stari.

Masina de stari implementata corespunde cu cea din figura de mai sus. Pentru o implementare fost definite enumeratiile:

```
enum states{START_S, WAIT_COMMAND, WAIT_DONE, WAIT_GIVE, WAIT_PLACEMENT};
enum states state=START_S;

enum cards{EXPLODE, DEFUSE, ATTACK, SKIP, FAVOR, SHUFFLE, SEE_FUTURE,
           TACO_CAT, POTATO_CAT, RAINBOW_CAT, BEARD_CAT, CATTERMELON};

const char* cards[12]={
    "EXPLODE",
    "DEFUSE",
    "ATTACK",
    "SKIP",
    "FAVOR",
    "SHUFFLE",
    "SEE_FUTURE",
    "TACO_CAT",
    "POTATO_CAT",
    "RAINBOW_CAT",
    "BEARD_CAT",
    "CATTERMELON"
};
```

Starea cartilor este retinuta in array-uri de intregi, identitatea cartii fiind corelata cu valorile c

```
int deck[50]={};
int player1[30]={}, player2[30]={};

int deckSize=0, player1Size=0, player2Size=0;

int currentPlayer;
int nrOfTurns;

enum players{p11, p12};
char *playersFiles[]={ "p11.txt", "p12.txt" };
char *playerNames[]={ "Player1", "Player2" };

int currentPlayedCard, chosenCard;
```

Implementarea logicii jocului a fost facuta cu urmatoarele functii:

```
void start_action();
void state_machine(int command);
void writeCardsToFile();
void deckCardsFile();
void futureFile();
void playCard();
void deleteCardFromCurrentPlayer(int cardIndex);
void insertCardToCurrentPlayer(int cardIndex);
int playerHasPairCards(int cardIndex);
int playerHasCard(int cardIndex);
int getCardIndex(char*);
```

```
void start_action();
```

- Implementeaza initializarea pachetului de carti si a cartilor din mana fiecarui jucator. Numarul de carti din fiecare tip sunt: 4-Defuse, 1-Explode, 4-(Attack, Skip, Favor, Shuffle, fiecare Useless ca o carte de future. Regulile de impartire sunt: Fiecare jucator va avea 8 carti, o singura carte de Defuse si o singura carte de Explode se afla in pachetul de pe "masa".

```
void state_machine(int command);
```

- Implementeaza actiunea fiecarei comenzi in functie de starea curenta a automatului. De exemplu, din starea de START_S singura comanda valida este START

```
void writeCardsToFile();
```

- Va scrie in fisierul jucatorului curent cartile din mana lui si va goli fisierul de carti a celui alaturat

```
void deckCardsFile();
```

- Functie de depanare, scrie intreg pachetul de pe masa in fisierul deckcards

```
void futureFile();
```

- Pentru jucarea cartii See Future, scrie in fisierul future cele 3 carti din varful stivei

```
void playCard();
```

- Implementeaza actiunea de joc a unei singure carti

```
void deleteCardFromCurrentPlayer(int cardIndex);
```

```
void insertCardToCurrentPlayer(int cardIndex);
```

- Sterge si respectiv insereaza o carte de tipul cardIndex (corespunzator cu enumeratia) in pachetul jucatorului curent

```
int playerHasPairCards(int cardIndex);
```

```
int playerHasCard(int cardIndex);
```

- Verifica daca jucatorul curent are respectiva/perechea de carti in pachetul propriu

```
int getCardIndex(char*);
```

- Functie utilitara, cauta indexul din enumeratie a cartii cu denumirea primita in sirul de caractere

• Capturi de ecran:

See future file:

- 1 Top 3 cards on the deck:
- 2 TACO CAT
- 3 EXPLODE
- 4 SEE FUTURE

Exemplu de joc:

```
START
CURRENT PLAYER:Player1
Player1, look at your cards in file pl1.txt and give a command
>EXTRACT
Player2, look at your cards in file pl2.txt and give a command
>PAIR RAINBOW CAT
Player2, you received SHUFFLE card from the another pack INDEX 5
Player2, look at your cards in file pl2.txt and give a command
>PLAY ATTACK
CURRENT PLAYER:Player1
Player1, look at your cards in file pl1.txt and give a command
>EXTRACT
Player1, look at your cards in file pl1.txt and give a command
>EXTRACT
The extracted card was EXPLODE
You had a DEFUSE in your hand
Write in the place.txt where to place the EXPLODE card and then say CHOOSE
  0 - will place it at bottom
 27 - will place it at top
  Between will place it in the middle. Values outside the interval will be clamped.
>CHOOSE
Player2, look at your cards in file pl2.txt and give a command
>PLAY SKIP
CURRENT PLAYER:Player1
Player1, look at your cards in file pl1.txt and give a command
>EXTRACT
The extracted card was EXPLODE
GAME OVER - Player2 wins!
```


1. Introducere

Unul dintre obiectivele unui programator este de a scrie un cod cât mai ușor de înțeles de către ceilalți, limbajul Whitespace fiind unul care îngreunează acest scop, deoarece programele scrise par a fi doar niște fișiere goale care nu conțin funcționalități. Astfel, posibilitatea de a crea programe „fantomă” a fost un motiv de a alege această temă în vederea realizării proiectului.

2. Descrierea proiectului

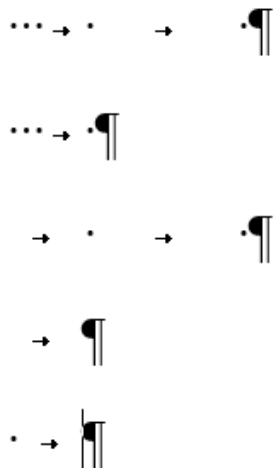
În cadrul acestui proiect se dorește implementarea funcționalităților limbajului Whitespace folosind Lex și Yacc. Acest limbaj are ca structură doar trei caractere albe, și anume spațiu, tab și linia nouă, restul caracterelor fiind ignorate. Limbajul funcționează numai cu numere întregi, acestea fiind reprezentate sub formă binară, utilizându-se spațiul pentru cifra 0, și tab pentru cifra 1. Primul caracter din cadrul unui număr reprezintă semnul acestuia. În cazul specificării unei etichete, nu se mai utilizează caracterul de semn, ci se începe direct cu numărul asociat.

Se permit realizarea operațiilor din patru mari categorii: operații cu stiva, operații aritmetice, operații pentru controlul fluxului programului și operații de intrare/ieșire. Astfel, în cadrul operațiilor aritmetice se vor implementa operațiile de adunare, scădere, înmulțire, împărțire de întregi și modulo. Majoritatea acestor operații se vor realiza asupra numerelor stocate în cadrul unei stive, astfel este nevoie de operații de punere și scoatere de pe stivă. Ca și operații de control a fluxului, se va putea plasa o anumită etichetă în cadrul programului, putând să se realizeze un salt la etichetă atât necondiționat, cât și în cazul în care pe vârful stivei se află numărul 0 sau un număr negativ. Pentru a facilita lucrul cu acest limbaj de programare, se implementează operații de intrare/ieșire, putând să se scrie de pe stivă un întreg sau un caracter.

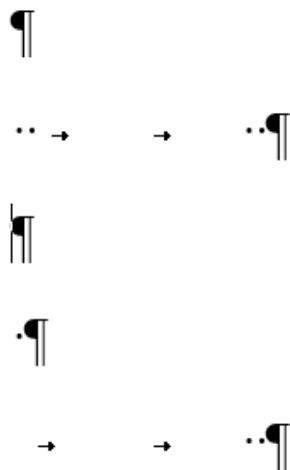
A doua funcționalitate este scrierea string-ului ltf:

```
....→ → → . → ..¶
....→ → .. → → ¶
....→ → . → → ..¶
..¶
..¶
..¶
```

Următorul exemplu ilustrează împărțirea numerelor 10 și 2, și afișarea câtului:



Ultimul exemplu ilustrează plasarea etichetei 12, și realizarea unui salt necondiționat la această etichetă:



3. Detalii implementare

Acest interpretator s-a realizat folosindu-se lex și yacc. În lex s-au identificat tokeni necesari, iar în yacc s-au parsat instrucțiunile limbajului. În continuare sunt prezentați tokeni folosiți alături de producții:

Tokeni: space, tab , lf

Producții:

line -> line instruction lf

instruction -> stackFuncion

instruction -> arithmeticFunction

instruction -> flowFunction

instruction -> IOFunction

stackFunction -> space stackCommand

stackCommand -> space number - punere pe stivă

stackCommand -> lf lf - scoatere de pe stivă

arithmeticFunction -> tab space arithmeticCommand

arithmeticCommand -> space space - adunare

arithmeticCommand -> space tab - scădere

arithmeticCommand -> space lf - înmulțire

arithmeticCommand -> tab space - împărțire

arithmeticCommand -> tab tab - modulo

flowFunction -> lf flowCommand

flowCommand -> space space label - marcare etichetă

flowCommand -> space lf label - salt necondiționat la etichetă

flowCommand -> tab space label - salt la etichetă dacă pe stivă e zero

flowCommand -> tab tab label - salt la etichetă dacă pe stivă e un număr negativ

IOFunction -> tab lf ioCommand

ioCommand -> space space - afișare caracter de pe stivă

ioCommand -> space tab - afișare număr de pe stivă

number -> sign value

sign -> space | tab

value -> value space | tab

label -> label space | tab

Pe partea de cod C, au fost necesare funcții și o structură pentru manipularea stivei, alături de funcții care pot realiza conversia unui șir de numere din baza doi într-un număr din baza 10.

Structură pentru stivă:

```
struct stack
```

```
{
```

```
    int maxsize;
```

```

int top;

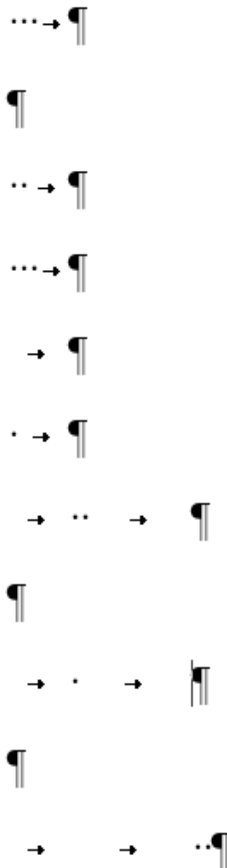
int *items;

};

```

4. Rezultate

În continuare este prezentat rezultatul unui program care pune pe stivă valoarea 1, plasează eticheta numărul 1, plasează pe stivă numărul 1, realizează o operație de scădere, și realizează un salt la eticheta cu numărul 1 în cazul în care pe stivă se află valoarea 0.



```

(base) dani@DESKTOP-GJC0II8:/mnt/d/Facultate-An3/Sem2/Limbaje formale si translatoare/Laborator/Proiect$ yacc -d whitespace.y
(base) dani@DESKTOP-GJC0II8:/mnt/d/Facultate-An3/Sem2/Limbaje formale si translatoare/Laborator/Proiect$ gcc lex.yy.c y.tab.c -ll -ly
(base) dani@DESKTOP-GJC0II8:/mnt/d/Facultate-An3/Sem2/Limbaje formale si translatoare/Laborator/Proiect$ ./a.out < input.in
11(base) dani@DESKTOP-GJC0II8:/mnt/d/Facultate-An3/Sem2/Limbaje formale si translatoare/Laborator/Proiect$ yacc -d whitespace.y
(base) dani@DESKTOP-GJC0II8:/mnt/d/Facultate-An3/Sem2/Limbaje formale si translatoare/Laborator/Proiect$ gcc lex.yy.c y.tab.c -ll -ly
(base) dani@DESKTOP-GJC0II8:/mnt/d/Facultate-An3/Sem2/Limbaje formale si translatoare/Laborator/Proiect$ ./a.out < input.in
11(base) dani@DESKTOP-GJC0II8:/mnt/d/Facultate-An3/Sem2/Limbaje formale si translatoare/Laborator/Proiect$ ./a.out < input.in

```

Bibliografie

1. <https://web.archive.org/web/20150618184706/http://compsoc.dur.ac.uk/whitespace/tutorial.php>

Contents

Cerința proiectului:
Explicațiile proiectului:
Arborele de parsare:
Structura codului:.....
Bibliografie:

Cerința proiectului:

Crearea unui translator din PL-SQL în limbaj natural. La intrare se dă o interogare în PL-SQL, iar la ieșire se generează ”traducerea” în limbaj natural.

Explicațiile proiectului:

Limbajele pe care l-am ales pentru realizarea translatorului au fost: Lex și Yacc.

Partea de creare și identificare a token-ilor a fost realizată în Lex. Mai specific: Avem instrucțiunea:

```
SELECT table_name.  
  
CASE owner  
  
    WHEN 'SYS' THEN The_owner_is_SYS;  
  
    WHEN 'SYSy' THEN The_owner_is_tampit;  
  
    ELSE 'The_owner_is_another_value';  
  
END  
  
FROM all_tables;
```

Cuvintele SELECT, CASE, WHEN, THEN, ELSE, END și FROM, au fost identificate individual și trimise prin clauza return către parser, care este reprezentat de fișierul Yacc.

```
("CASE" | "case") {return CASE;}
```

Toate celelalte cuvinte care nu reprezintă KeyWord-uri specifice PL-SQL, au fost recunoscute conform următorului regex:

```
[a-zA-Z0-9\.\(\)\-\_\'\\"%\\*\+\/\=\:\|\\>]+
```

Ca mai apoi să fie trimise către fișierul de tip Yacc prin yylval.word. yylval este o variabila de tip int, care trimite date de Lex la Yacc. În situația dată, noi doream să trimitem și să anexăm token-ului WORD, un șir de caractere, așa ca i-am atribuit lui yylval word, care este declarat într-o structură de date de tip union în fișierul Yacc:

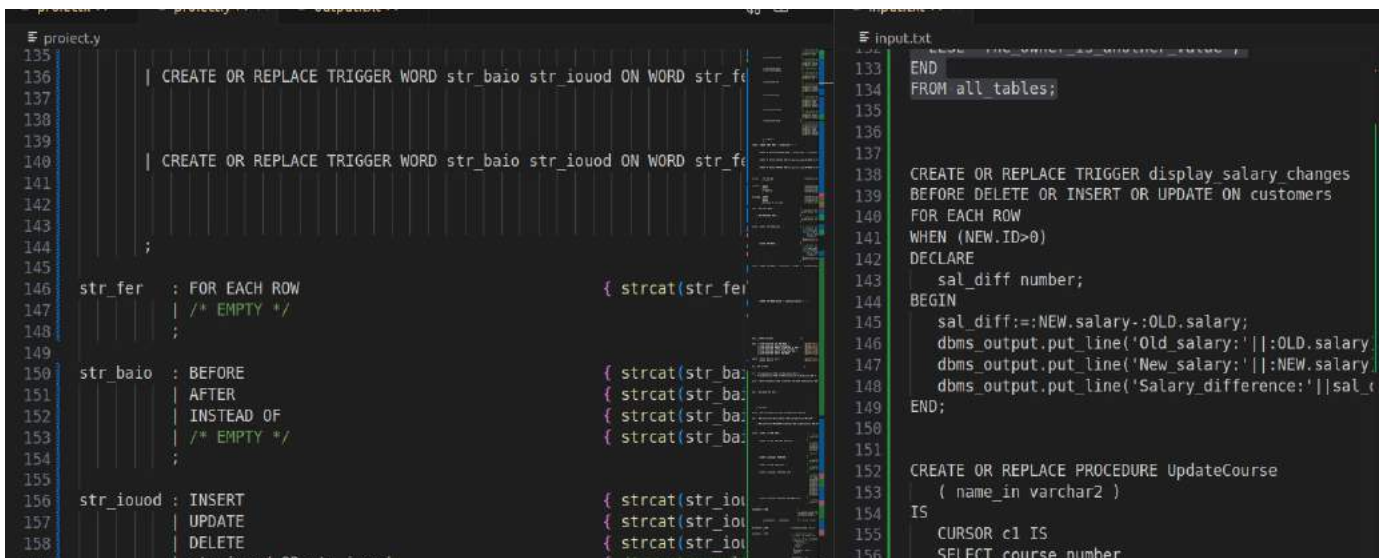
```
%union {char *word;}
```

Pentru a putea utiliza C și funcțiile aferente acestuia, am importat librăriile stdio.h, stdlib.h și string.h (pentru lucrul cu șiruri de caractere). De asemenea, am utilizat funcția

substr() ¹ pentru a elimina ghilimelele(") sau apostroafele(') care încadrează string-uri specifice (cum ar fi numele unei persoane).

Pe lângă token-ii care conțin keyWord-uri, am trimis și token-i ca semne de punctuație, precum *, care în clauza SELECT reprezintă ideea de ALL (afișarea tuturor coloanelor din tabel).

Ca și parser s-a utilizat Yacc. Acesta preia token-ii din gramatică (care sunt simboluri terminale) și cu ajutorul lor creează producții (sau reguli) prin care recunoaște sintaxa limbajului de baze de date PL-SQL. Pentru crearea unor producții cât mai generale, acesta utilizează simboluri non-terminale care conțin ramificații pentru ceea ce reprezintă acesta. A se vedea formatul unui trigger, unde poate conține la început unul din cele 3 cuvinte: BEFORE, AFTER sau INSTEAD OF:



The screenshot shows a code editor with two files: 'proiect.y' and 'input.txt'. The 'proiect.y' file contains PL-SQL code for creating triggers and a procedure. The 'input.txt' file contains the same code. The code is as follows:

```
135 | CREATE OR REPLACE TRIGGER WORD str_baio str_iouod ON WORD str_fe
136 |
137 |
138 |
139 | CREATE OR REPLACE TRIGGER WORD str_baio str_iouod ON WORD str_fe
140 |
141 |
142 |
143 |
144 |
145 |
146 str_fer : FOR EACH ROW
147 | /* EMPTY */
148 |
149 |
150 str_baio : BEFORE
151 | AFTER
152 | INSTEAD OF
153 | /* EMPTY */
154 |
155 |
156 str_iouod : INSERT
157 | UPDATE
158 | DELETE
159 | str_iouod OR str_iouod

133 END
134 FROM all tables;
135
136
137
138 CREATE OR REPLACE TRIGGER display_salary_changes
139 BEFORE DELETE OR INSERT OR UPDATE ON customers
140 FOR EACH ROW
141 WHEN (NEW.ID>0)
142 DECLARE
143 sal_diff number;
144 BEGIN
145 sal_diff:=NEW.salary-:OLD.salary;
146 dbms_output.put_line('Old salary:'||:OLD.salary);
147 dbms_output.put_line('New salary:'||:NEW.salary);
148 dbms_output.put_line('Salary difference:'||sal_diff);
149 END;
150
151
152 CREATE OR REPLACE PROCEDURE UpdateCourse
153 ( name_in varchar2 )
154 IS
155 CURSOR c1 IS
156 SELECT course number
```

Simbolul de start (marcat cu %start) este instruction. Tokenii sunt marcați prin simbolul %token, declarați ca având același tip de date ca și word din union (adică char*), iar simbolurilor non-terminale le sunt asociate prin %type tipul de date al lui word.

```
%start instruction
%union {char *word;}
%token <word> CREATE DROP DELETE INSERT INTO VALUES WHERE TABLE WORD AND OR
ALTER COLUMN MODIFY RENAME ADD TO UPDATE IF THEN END SET EQ LTE GTE NQ ELSE
FOR LOOP REVERSE IN SELECT FROM ALL PROCEDURE IS EXCEPTION OTHERS BEGINN
REPLACE WHEN WHILE CASE TRIGGER OF AFTER INSTEAD EACH DECLARE ON ROW BEFORE
CURSOR COMMIT CLOSE OPEN FETCH
%type <word> wordsCreate
%type <word> wordsWhere
```

După identificarea structurii exemplului de PL-SQL, se generează traducerea utilizând ca limbaj de programare C, mai precis lucrul pe string-uri.

Mai multe detalii se află în comentariile din interiorul codului ².

¹ Această funcție a fost preluată de pe internet, de pe site-ul: <https://www.techiedelight.com/implement-substr-function-c/>

² A se vedea fișierul proiect.y

Se poate observa existența unor conflicte, care nu au fost reparate, pentru că Yacc-ul are un mecanism specific de tratare al acestora. Spre exemplu, dacă este vorba despre conflicte de tip shift/reduce, acesta face alege să facă shift.

Arborele de parsare:

Prin intermediul arborelui de parsare s-a analizat adâncimea lui și producțiile prin care a trecut pentru a fi tradus următorul exemplu:

```
CREATE OR REPLACE PROCEDURE UpdateCourse

( name_in varchar2 )

IS

CURSOR c1 IS

SELECT course_number

FROM courses_tbl

WHERE course_name = name_in;;

BEGIN

OPEN c1;

FETCH c1 INTO cnumber;

IF c1%notfound THEN

cnumber:=9999;

END IF;

INSERT INTO student_courses

( course_name, course_number )

VALUES

( name_in, cnumber );

COMMIT;

CLOSE c1;

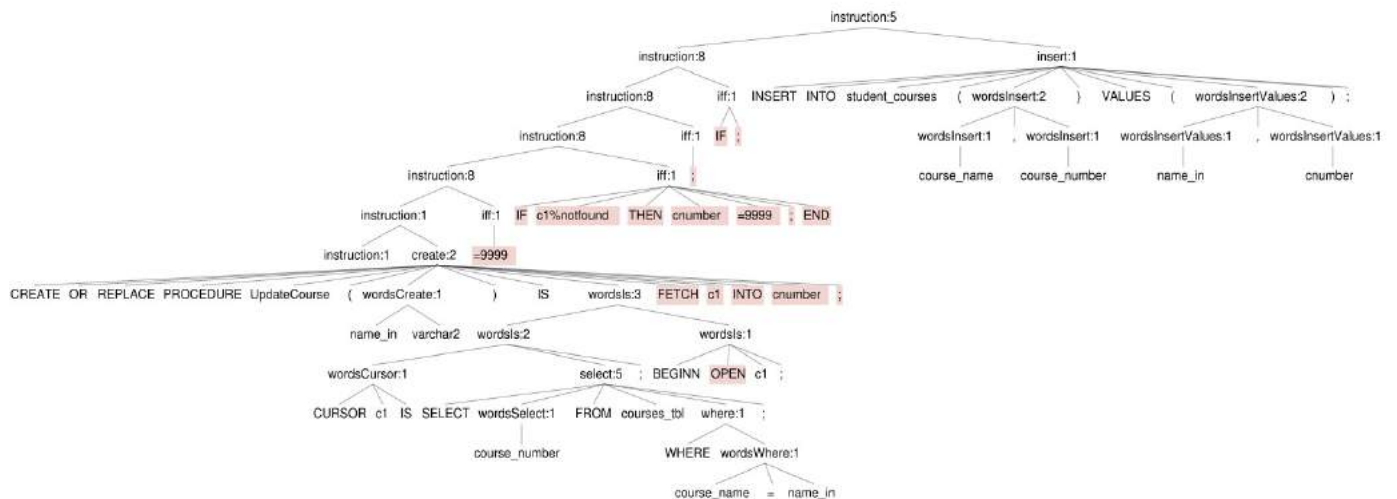
EXCEPTION

WHEN OTHERS THEN

raise_application_error;

END;
```


Acesta este arborele de parsare generat:



Structura codului:

Codul a fost construit pe modelul oferit în cadrul lucrărilor de laborator ³ și respectând regulile de bune practici sugerate de acestea.

³ Lucrările de laborator 4-8: <https://moodle.cs.utcluj.ro/course/view.php?id=569> – site-ul lor

Bibliografie:

1. <https://www.techiedelight.com/implement-substr-function-c/>
2. <https://moodle.cs.utcluj.ro/course/view.php?id=569>
3. <http://labantlr.org/> - pentru arborele de parsare

1. Scurtă introducere

Acest proiect urmărește dezvoltarea unui chatbot care ghidează un om să își găsească piesa sau piesele necesare care trebuie schimbate la mașină. Așadar, prin intermediul proiectului o persoană poate identifica componenta necesară care trebuie înlocuită la respectivul autovehicul.

2. Descrierea proiectului

Discuția cu bot-ul se va realiza într-un chat, asemenea unui terminal, unde persoana va pune întrebări sau va da detalii despre problema pe care o prezintă mașina, iar chatbot-ul în funcție de respectivele răspunsuri va ști ce piesă să recomande.

Posibilele piese care se recomandă vor fi stabilite clar dintr-un set de componente și, de asemenea, chatbot-ul va ști să schimbe sau să ia decizii bazate nu doar pe ultimul răspuns al utilizatorului.

3. Exemplu flow execuție

Hei, I have a problem.

Is the problem digital or mechanical?

Is digital one, what I have to buy?

What are the symptoms?

When I'm trying to connect my phone at car, it doesn't work well.

What smartphone do you have?

I have brake problems.

This is not a digital problem. Going back to problem types. Is the problem digital or mechanical?

Mechanics.

What are the symptoms?

When I'm braking the car makes some noise.

When you brake you hear a long or short sound?

It's a long sound.

The problem are brakes. You have to buy brake pads.

Thanks a lot!

My pleasure, good bye!

4. Implementare

Chatbot-ul va răspunde utilizatorului ce componentă trebuie să cumpere. Componentele sunt reprezentate de o structura

```
typedef struct Component {  
    char name[30];  
    char type[10];  
    int nr_pieces;  
    double price;  
} component;
```

Îmi populez un vector de structuri cu cateva componente pentru problemele posibile, din care bot-ul va recomanda o piesă utilizatorului la finalul comunicării.

Dacă utilizatorul nu scrie o problema corespunzătoare pentru tipul problemei pe care l-a ales initial (digital, dar scrie o problema mecanică sau invers) va primi răspunsul că nu e o problemă pe care a specificat-o și va reveni la început, la alegerea problemei. Dacă userul dă un răspuns valid va fi afișat răspunsul : Invalid answer! Answer again!

5. Functionare

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
Hei, I have a problem with my car
Is the problem digital or mechanical?
Is digital one
What are the symptoms?
Can't connect phone to car
What smartphone do you have?
Iphone
The problem is the bluetooth sensor
You have to buy: bluetooth ios sensor, 1 piece(s) necessary at 440.20$ price.
Thanks
My pleasure, good bye!
```

```
Hei, I have a problem
Is the problem digital or mechanical?
It's mechanics
What are the symptoms?
Braking
When you brake you hear a long or short sound?
Not long
The problem are brakes
You have to buy: brake discs, 2 piece(s) necessary at 341.50$ price.
Thanks
My pleasure, good bye!
```

CUPRINS

1. Introducere
2. Instrumente utilizate
3. Introducere CLU (LUIS)
4. Descrierea implementarii in CLU (LUIS)
5. Descrierea implementarii aplicatiei utilizator
6. Descrierea functionalitatilor aplicatiei

1. Introducere

Smart Home Controller este o aplicație inteligentă pentru controlul și gestionarea sistemelor și dispozitivelor dintr-o locuință inteligentă. Această aplicație are ca scop facilitarea și automatizarea controlului asupra diferitelor aspecte ale unei case inteligente, cum ar fi iluminatul, încălzirea, ventilația, sistemele de securitate și multe altele. Prin intermediul unei interfețe ușor de utilizat pe smartphone sau pe alte dispozitive inteligente, utilizatorii pot controla și monitoriza toate dispozitivele și sistemele conectate la rețeaua lor inteligentă, oferind o experiență de viață mai comodă, sigură și eficientă energetic.

O casă inteligentă se bazează pe tehnologia Internet of Things (IoT) pentru a conecta și a controla diferite dispozitive și sisteme din interiorul casei. Smart Home Controller vine în întâmpinarea nevoilor utilizatorilor de a gestiona și controla aceste dispozitive și sisteme într-un mod eficient și simplu. Prin intermediul aplicației, utilizatorii pot avea acces rapid și ușor la funcționalitățile caselor lor inteligente, precum ajustarea temperaturii, aprinderea și stinsul luminilor, verificarea camerelor de supraveghere sau activarea alarmei de securitate. Această aplicație aduce confort, economie de timp și resurse, precum și o experiență îmbunătățită a vieții de zi cu zi.

Smart Home Controller este potrivit pentru orice tip de locuință sau clădire inteligentă, de la apartamente și case rezidențiale la birouri și spații comerciale. Poate fi utilizată de către proprietarii de case inteligente, chiriași sau manageri de clădiri pentru a controla și monitoriza eficient toate aspectele unei locuințe inteligente. Aplicația poate fi adaptată pentru a se potrivi cu diferite nevoi și preferințe, oferind posibilitatea de a personaliza și configura scenarii și automatizări în funcție de preferințele și rutina fiecărui utilizator.

2. Instrumente utilizate

Motivația utilizării tool-ului LUIS Azure pentru înțelegerea comenzilor:

Utilizarea tool-ului LUIS Azure pentru înțelegerea comenzilor aduce multiple beneficii în dezvoltarea unei aplicații cu interfață vocală sau bazată pe text. LUIS (Language Understanding Intelligent Service) oferă funcționalități puternice de înțelegere a limbajului natural, permițând aplicației să recunoască și să interpreteze comenzile și intențiile utilizatorului într-un mod precis și eficient. Prin utilizarea algoritmilor de învățare automată și a modelelor predefinite de limbaj, LUIS poate fi antrenat pentru a recunoaște diverse forme și variante de comenzi, facilitând interacțiunea fluidă și naturală cu utilizatorii. Utilizând LUIS Azure, dezvoltatorii pot economisi timp și efort în dezvoltarea și optimizarea funcționalității de înțelegere a limbajului natural, oferind o experiență îmbunătățită utilizatorilor și creând aplicații mai inteligente și mai intuitive.

Motivația utilizării Postman pentru testarea API-ului:

Postman este un tool popular și puternic pentru testarea și gestionarea API-urilor. Utilizarea Postman aduce multiple avantaje în procesul de dezvoltare și testare a funcționalității API-ului unei aplicații. Prin intermediul Postman, dezvoltatorii pot crea și executa solicitări HTTP personalizate către API, testându-l în diverse scenarii și evaluând comportamentul acestuia în timp real. Postman oferă o interfață intuitivă și ușor de utilizat, permițând dezvoltatorilor să exploreze și să testeze rapid diferite rute și parametri ale API-ului. Prin utilizarea funcționalităților de testare automate și de monitorizare, Postman facilitează identificarea și remedierea rapidă a erorilor și problemelor de performanță ale API-ului. Utilizarea Postman în procesul de dezvoltare asigură o calitate superioară a API-ului și oferă un mediu de testare robust și fiabil pentru integrarea cu alte componente ale aplicației.

Motivația utilizării HTML pentru pagina web a aplicației:

HTML (HyperText Markup Language) este limbajul de marcare standard utilizat pentru construirea paginilor web. Utilizarea HTML în construirea paginii web a aplicației aduce numeroase avantaje în ceea ce privește flexibilitatea, compatibilitatea și ușurința de utilizare. HTML oferă o structură și o organizare clară a elementelor și conținutului paginii web, permițând dezvoltatorilor să creeze o interfață intuitivă și ușor de navigat pentru utilizatori. Prin intermediul tagurilor și atributelor HTML, dezvoltatorii pot adăuga funcționalități interactive, stiluri și formatare pentru a crea o experiență vizuală plăcută și coerentă.

3. Introducere CLU (LUIS)

CLU Azure (Conversational Language Understanding Azure) este un serviciu puternic oferit de Microsoft Azure, care se concentrează pe înțelegerea și interpretarea limbajului natural în contextul conversațiilor. Acesta utilizează tehnici avansate de învățare automată și procesare a limbajului natural pentru a permite aplicațiilor și sistemelor să comunice și să interacționeze într-un mod natural cu utilizatorii umani.

CLU Azure este special conceput pentru a ajuta dezvoltatorii să construiască aplicații de chatbot-uri inteligente, asistenți virtuali și alte interfețe de conversație inteligente. Prin intermediul acestui serviciu, aplicațiile pot înțelege intențiile utilizatorilor, recunoaște entități și extrage informații relevante din conversații.

CLU Azure permite antrenarea și personalizarea modelelor de înțelegere a limbajului natural pentru a se potrivi cu necesitățile specifice ale aplicațiilor. Dezvoltatorii pot defini intenții, entități și exemple de dialog pentru a antrena și optimiza modelele. De asemenea, serviciul oferă funcționalități pentru gestionarea și monitorizarea conversațiilor, permițând dezvoltatorilor să îmbunătățească continuu performanța și precizia înțelegerii limbajului natural.

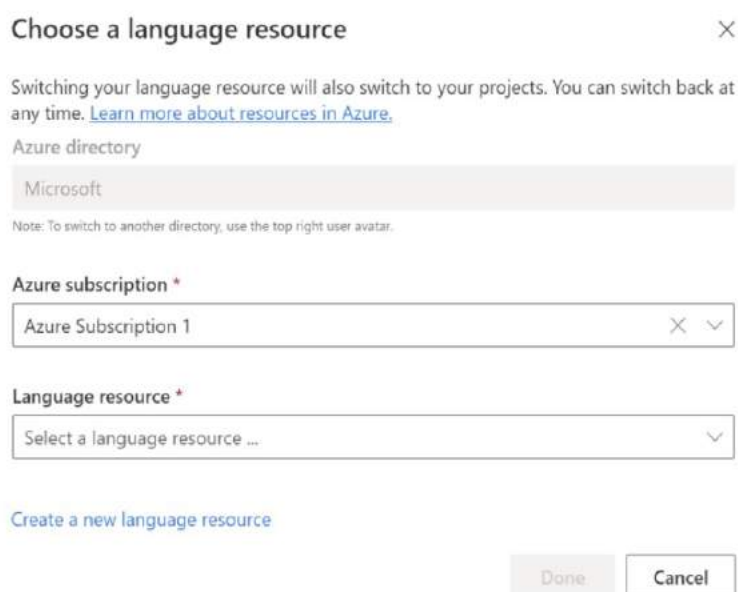
CLU Azure vine cu o serie de funcționalități și servicii suplimentare, precum integrarea cu alte servicii Azure, cum ar fi Speech to Text (convertirea vorbirii în text) și Text to Speech (convertirea textului în vorbire), pentru a crea experiențe de conversație bogate și interactive.

Cu ajutorul CLU Azure, dezvoltatorii pot crea aplicații de conversație mai inteligente, care pot interacționa în mod natural și eficient cu utilizatorii, oferind astfel o experiență conversațională superioară și îmbunătățind eficiența și satisfacția utilizatorilor.

4. Descrierea implementarii in CLU (LUIS)

Pentru implementarea aplicatiei in CLU (conversational language understanding) este necesara crearea unui cont pe platforma Microsoft Azure.

Urmatorul pas este logarea pe platforma Language Studio, provider Microsoft. In timpul logarii, apare o fereastra numita “Choose a language resource” – in cazul nostru, nu am avut disponibila o resursa de limba, astfel, a fost necesara crearea uneia noi:



Choose a language resource

Switching your language resource will also switch to your projects. You can switch back at any time. [Learn more about resources in Azure.](#)

Azure directory

Microsoft

Note: To switch to another directory, use the top right user avatar.

Azure subscription *

Azure Subscription 1

Language resource *

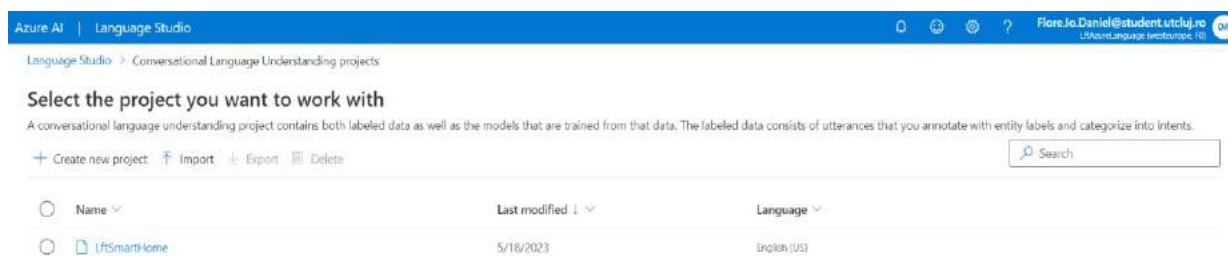
Select a language resource ...

[Create a new language resource](#)

Done Cancel

Dupa crearea si selectarea noii resurse de limba, este necesara crearea unui proiect conversational language understanding. Un astfel de proiect reprezinta o arie de munca pentru construirea unor modele de machine learning bazate pe datele introduce de noi. Proiectul nou creat a fost distribuit intre noi – ambii student avand access la editarea sa.

4.1. Proiect



Azure AI | Language Studio

Language Studio > Conversational Language Understanding projects

Select the project you want to work with

A conversational language understanding project contains both labeled data as well as the models that are trained from that data. The labeled data consists of utterances that you annotate with entity labels and categorize into intents.

+ Create new project Import Export Delete

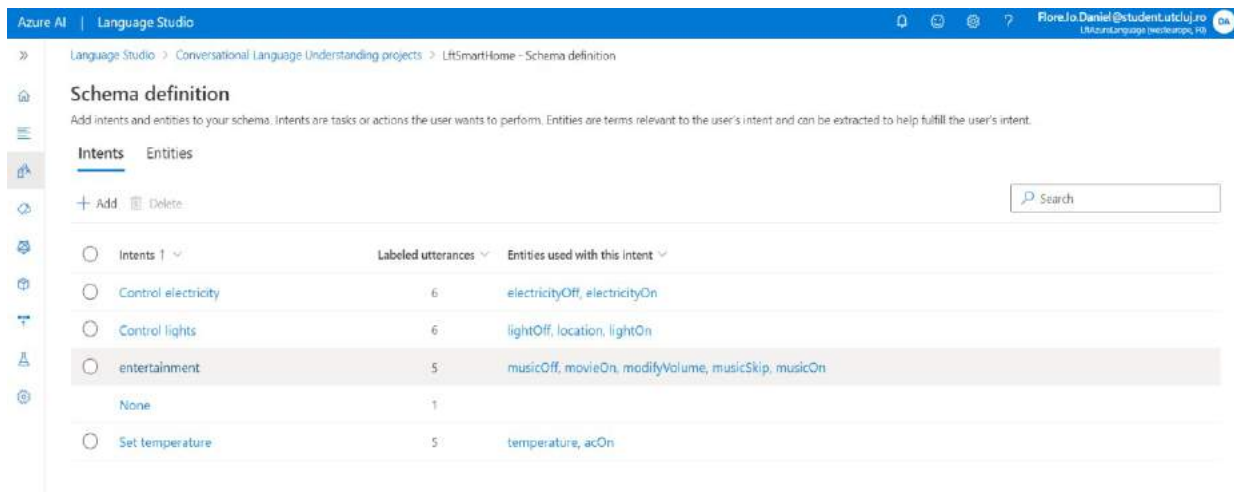
Search

Name	Last modified	Language
LftSmartHome	5/18/2023	English (US)

4.2. Intends & Entities

Lista cu intentiile generale ale utilizator

- Control electricity
- Control lights
- Entertainment
- Set temperature
- None



Pentru fiecare dintre aceste intentii, au fost declarate posibile mesaje tip input pe care aplicatia le poate primi din partea utilizatorilor:

Control electricity	Turn off <u>electricity</u> in the house. electricityOff	Control lights	Turn off the <u>living room</u> lights. lightOff location
Control electricity	Turn on <u>electricity</u> in the house. electricityOn	Control lights	Turn the <u>bedroom</u> lights off. lightOff locat...
Control electricity	Turn on <u>home's</u> electicity. electricit...	Control lights	Turn on all the lights in the house. lightOn loc...
Control electricity	Turn on <u>power</u> to the entire house. electricity...	Control lights	Turn off all the lights in the house. lightOff loc...
Control electricity	Turn off <u>power</u> to the entire house. electricity0...	Control lights	Turn the <u>bedroom</u> lights on. lightOn locat... lightOn
Control electricity	Turn off <u>home's</u> electicity. electricity...	Control lights	Turn on the <u>living room</u> lights. lightOn location light...

UNIVERSITATEA TEHNICA DIN CLUJ-NAPOCA

Departamentul Calculatoare
Facultatea de Automatica si Calculatoare

Set temperature	What's the <u>current temperature?</u> temperature	entertainment	<u>Pause the music</u> in the bedroom. musicOff
Set temperature	<u>Lower the heat</u> in the bedroom. acOn	entertainment	<u>Play a movie</u> on the living room TV. movieOn
Set temperature	<u>Turn on the air</u> conditioning. acOn	entertainment	<u>Set the volume</u> to 50%. modifyVolume
Set temperature	<u>Set the thermostat</u> to 30 degrees. acOn	entertainment	<u>Skip this song.</u> musicSkip
Set temperature	<u>Increase the temperature</u> by 2 degrees. acOn	entertainment	<u>Play some music.</u> musicOn

Pentru fiecare posibil input primit de la utilizator, au fost declarate entitati – Entity – pentru recunoasterea anumitor secvente de text folosite pentru identificarea intentiei utilizatorului. Aceste entitati pot fi vizibile in lista de mai jos:

Azure AI Language Studio			
Language Studio > Conversational Language Understanding projects > LiftSmartHome - Schema definition			
Schema definition			
Add intents and entities to your schema. Intents are tasks or actions the user wants to perform. Entities are terms relevant to the user's intent and can be extracted to help fulfill the user's intent.			
Intents Entities			
+ Add Edit entity components Delete			
	Entities	Labeled in uttera...	Used in intents
	musicOn	1	entertainment
	musicSkip	1	entertainment
	movieOn	1	entertainment
	modifyVolume	1	entertainment
	musicOff	1	entertainment
	temperature	1	Set temperature
	acOn	4	Set temperature
	location	6	Control lights
	lightOn	5	Control lights
	lightOff	3	Control lights

4.3. Antrenarea modelului

Dupa introducerea intend-urilor, entity-urilor, respectiv a inputurilor necesare functionarii aplicatiei, se realizeaza o etichetare a inputurilor cu etichetele specifice fiecarei intentii. Totodata, pentru a obtine o aplicatie functionala, este necesara antrenarea treptata a modelului cu datele curente, pentru a avea raspunsuri eficiente.

Pentru antrenarea modelului, se selecteaza optiunea de meniu – Training jobs – si se completeaza cu datele dorite:

The screenshot displays the 'Start a training job' dialog in the LftSmartHome application. The interface is in Romanian and shows the following sections:

- Start a training job**: The main title of the dialog.
- Select Model**: A section explaining that users can train a new model or overwrite an existing one. It notes that training a new model is best at the beginning or for comparing performances, and that overwriting will replace the old model with the new data.
- Training configuration**: A message stating the current version is the latest 2022-09-01 training configuration version, with a link to [Click here to change](#).
- Train a new model**: A radio button option that is selected. Below it is a text input field containing 'test-training'.
- Overwrite an existing model**: An unselected radio button option.
- Training mode**: Two radio button options: 'Standard training (free)' (selected) and 'Advanced training'. The 'Standard training (free)' option is described as having faster training times for quicker iterations, only available in English, and free of charge. The 'Advanced training' option is described as using fine-tuned neural network transformer models and being able to train multilingual projects, with a link to [View pricing](#).
- Data splitting**: A section explaining that separate datasets are used to train the model and test its accuracy, with a link to [Learn more about data splitting](#). It contains two radio button options: 'Automatically split the testing set from training data' (selected) and 'Use a manual split of training and testing data'.
- Buttons**: At the bottom, there are two buttons: 'Train' (highlighted in blue) and 'Cancel'.

Urmatorul pas este analizarea si verificarea training job-ului, obtinerea unui status de complete ne garanteaza faptul ca modelul a fost antrenat cu noile date disponibile in cadrul proiectului nostru:

»

Language Studio > Conversational Language Understanding projects > LftSmartHome - Training jobs

Home

Menu

Project

Model

Training jobs

Deployment

Monitoring

Settings

Training jobs

After labeling your data, start a training job to create a model that understands conversational utterances. Select the trained model from a successful job to view its performance results. Jobs from the last 7 days are displayed here. [Learn about the pricing for training](#)

+ Start a training job

✕ Cancel

Search

	Training job id ▾	Status ▾	Output model ▾	Submitted ↓ ▾
☐	9967ae92-a205-41c2...	✔ Training succeeded	test-electricity	5/18/2023
☐	9f54b02c-a906-4970...	✔ Training succeeded	test-electricity	5/18/2023
☐	e8ea59f4-b2fd-4fda...	✔ Training succeeded	test-electricity	5/18/2023
☐	0eb6a73e-e3e3-45c...	✔ Training succeeded	test-electricity	5/18/2023

4.4. Deploy

Pentru a putea obtine raspunsuri specific datelor curente existente in aplicatie, este necesara operatia de deploy a modelului current. Astfel, dupa selectarea meniului – Deploying a model – trebuie create un nou deployment in care se va selecta un nou nume pentru deploy-ul current, respective un model antrenat la pasul anterior.

Add deployment

Create or select an existing deployment name
You can create a new deployment name or overwrite an existing deployment name to add a trained model to them

☒ Create a new deployment name
test-deploy

☐ Overwrite an existing deployment name
By selecting an existing deployment name, you will override any deployed model to this deployment name

Assign trained model to your deployment name
Add a trained model to your selected deployment name

Model
test-electricity

Deployment regions
Select the regions to deploy in based on the available assigned resources.

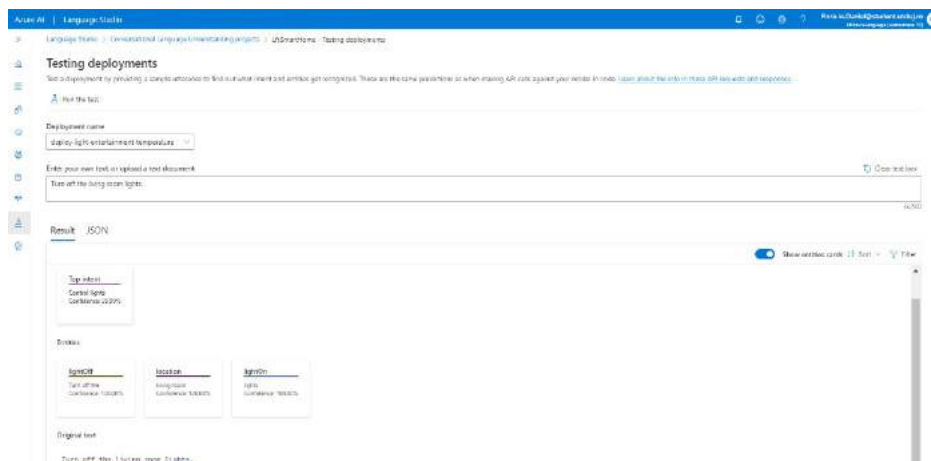
Regions
West Europe - LftAzureLanguage

Deploy **Cancel**

4.5. Testarea deploy-ului

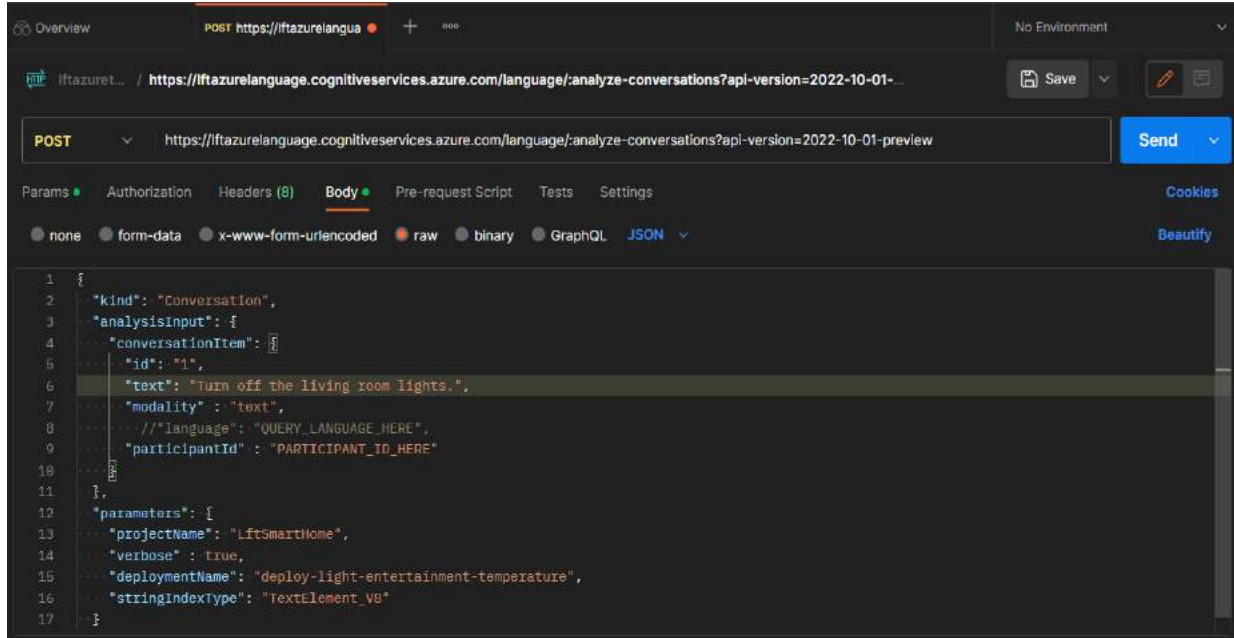
Pentru verificarea corectitudinii deploy-ului, este necesara testarea acestuia. Astfel, pentru testare am folosit tool-ul de testare integrat in Language Studio, dar si tool-ul Postman pentru testarea prin intermediul REST API Calls.

Selectam deploy-ul current propus pentru testare, introducem un text dorit si analizam raspunsul aplicatiei.

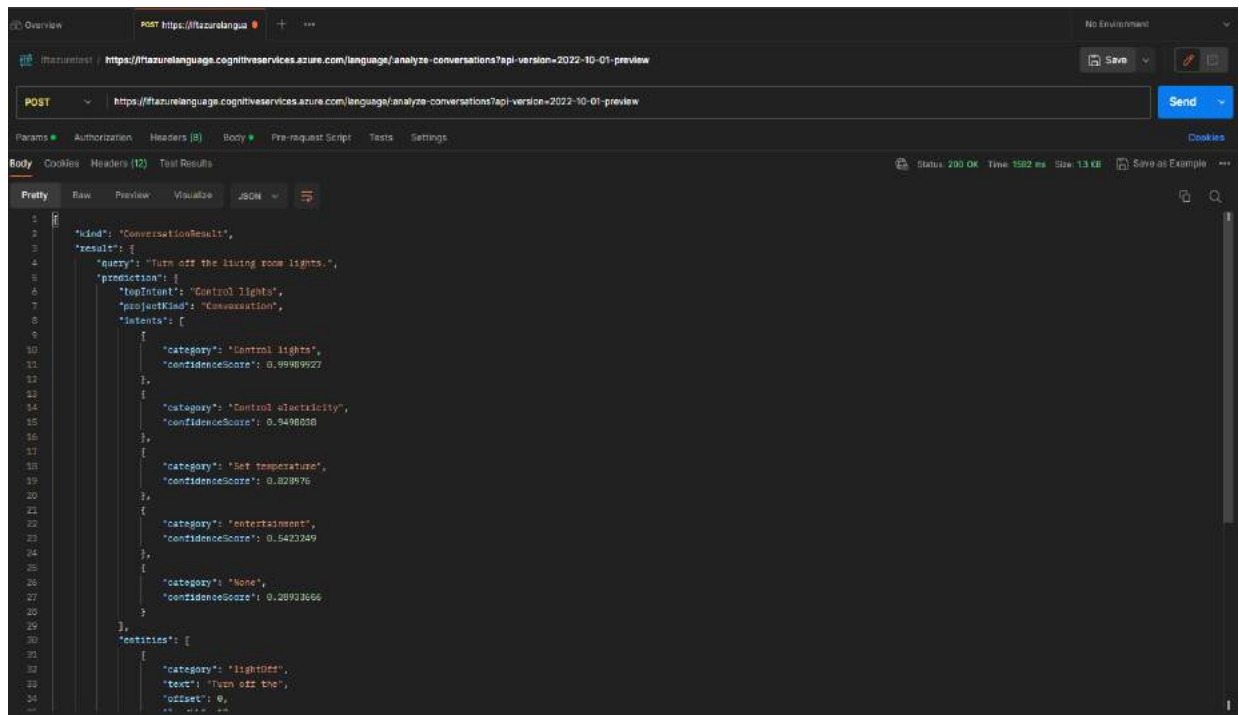


4.5.1 Testarea deploy-ului prin intermediul Postman

Configuram cererea: folosim o metoda POST, introducem link-ul catre proiect, client secretul in params si un request body cu urmatoorii parametrii:



Dupa apasarea butonului de Send, vom verifica response body-ul obtinut dupa REST API Call:



4.6. Structura REST API Call

Method type: POST

Link: <https://lftazurelanguage.cognitiveservices.azure.com/language/:analyze-conversations?api-version=2022-10-01-preview>

Params:

Key: api-version, Value: 2022-10-01-preview

Headers:

Key: Ocp-Apim-Subscription-Key, Value: <client secret>

Body:

```
{
  "kind": "Conversation",
  "analysisInput": {
    "conversationItem": {
      "id": "1",
      "text": "QUERY",
      "modality": "text",
      // "language": "QUERY_LANGUAGE_HERE",
      "participantId": "PARTICIPANT_ID_HERE"
    }
  },
  "parameters": {
    "projectName": "LftSmartHome",
    "verbose": true,
    "deploymentName": "deploy-light-entertainment-temperature",
    "stringIndexType": "TextElement_V8"
  }
}
```

5. Descrierea implementarii aplicatiei utilizator

Pentru proiectul prezentat, am ales sa folosim limbajul HTML, intrucat este limbajul potrivit pentru a realiza o aplicatie utilizator din următoarele motive:

Elemente de intrare și ieșire: HTML oferă o gamă variată de elemente de intrare și ieșire, cum ar fi input, button și textarea, care sunt esențiale pentru o interfață CLI. În aplicatia prezentata, putem observa prezenta unui câmp de introducere text (inputText), unui buton pentru trimiterea comenzilor (submitButton) și un câmp de text pentru afișarea rezultatelor (outputText).

Stilizare și prezentare: HTML poate fi stilizat cu ajutorul CSS (Cascading Style Sheets) pentru a oferi o prezentare atractivă și coerentă a interfeței. Datorita utilizarii CSS, am reusit sa cream o aplicatie usor de folosit, dar si cu un aspect atragator pentru utilizator.

Mai jos, se poate vizualiza partea grafica a aplicatiei.



Interactivitate: Prin utilizarea JavaScript în combinație cu HTML, am putut adăuga funcționalități interactive la interfața CLI. În acest sens, am putut adauga evenimente de clic la butonul submitButton pentru a apela o funcție JavaScript (makeApiCall) care efectuează o cerere către serviciul web din cadrul Microsoft Azure.

Integrare cu servicii web: Interfața CLI poate fi integrată cu servicii web externe pentru a obține date sau pentru a efectua acțiuni specifice. În codul furnizat mai sus, putem vedea

utilizarea unui serviciu web de analiză a conversațiilor LUIS (Language Understanding Intelligent Service) pentru a interpreta comenzile utilizatorului.

Ușurință de implementare și utilizare: HTML este un limbaj accesibil și ușor de înțeles. Sintaxa sa simplă și intuitivă face ca dezvoltarea și întreținerea interfeței CLI să fie mai ușoară. De asemenea, există o mulțime de resurse și documentație disponibile online care pot ajuta dezvoltatorii să învețe și să utilizeze HTML eficient.

În concluzie, HTML este limbajul potrivit pentru a implementa o interfață CLI pentru o casă inteligentă, deoarece oferă structură clară, elemente de intrare și ieșire, posibilități de stilizare și prezentare, interactivitate, integrare cu servicii web și o ușurință de implementare și utilizare.

6. Descrierea functionalitatilor aplicatiei

Acum ca am inteles cum a fost implementata aplicatia si care sunt elementele, vom descrie in continuare modul in care aplicatia poate fi folosita.

Pentru a utiliza aplicația CLI pentru casă inteligentă, utilizatorul ar trebui să urmeze următorii pași:

Accesarea aplicației: Utilizatorul ar trebui să deschidă un browser web și să acceseze adresa URL a aplicației CLI pentru casă inteligentă. Acest lucru ar putea fi realizat prin introducerea adresei URL în bara de adrese a browserului și apăsarea tastei Enter.

Interfața CLI: Odată accesată aplicația, utilizatorul va vedea interfața CLI pe ecran. Interfața CLI va include un câmp de introducere text (inputText) în care utilizatorul poate tasta comenzile și un câmp de text (outputText) în care vor fi afișate rezultatele sau mesajele de sistem.

Introducerea comenzilor: Utilizatorul ar trebui să introducă comenzile în câmpul de introducere text. Aceste comenzi pot fi de exemplu "aprindere lumina", "setare temperatura 22" sau "verificare starea ușii". Comenzile pot fi introduse într-un format specific, care este prevăzut în instrucțiunile aplicației sau în documentația asociată.

Transmiterea comenzilor: După ce utilizatorul a introdus o comandă în câmpul de introducere text, acesta ar trebui să apese butonul "Submit" (submitButton) pentru a trimite comanda către aplicație. Acest lucru va declanșa o acțiune în JavaScript care va prelua textul introdus în câmpul de introducere și va trimite o cerere către un serviciu web sau va procesa comanda direct în cadrul aplicației.

Procesarea comenzilor: Aplicația va prelua comanda introdusă de utilizator și va interpreta cererea. Aceasta poate implica apelarea unui serviciu web extern pentru a controla dispozitivele din casa inteligentă sau poate implica efectuarea unor acțiuni directe în cadrul aplicației, cum ar fi afișarea stării dispozitivelor sau modificarea setărilor.

Afișarea rezultatelor: După procesarea comenzii, rezultatele sau mesajele corespunzătoare vor fi afișate în câmpul de text (outputText) al interfeței CLI. Utilizatorul va putea vedea răspunsul la comanda introdusă și eventualele informații suplimentare.

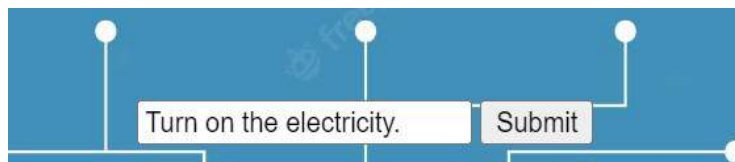
Continuarea interacțiunii: Utilizatorul poate continua interacțiunea cu aplicația prin repetarea pașilor 3-6 pentru a introduce și executa alte comenzi. Acest lucru permite utilizatorului să controleze și să monitorizeze dispozitivele și funcțiile din casa inteligentă folosind interfața CLI.

Mai jos, se pot vedea acești pași ilustrați, pentru a înțelege exact modul în care aplicația funcționează.

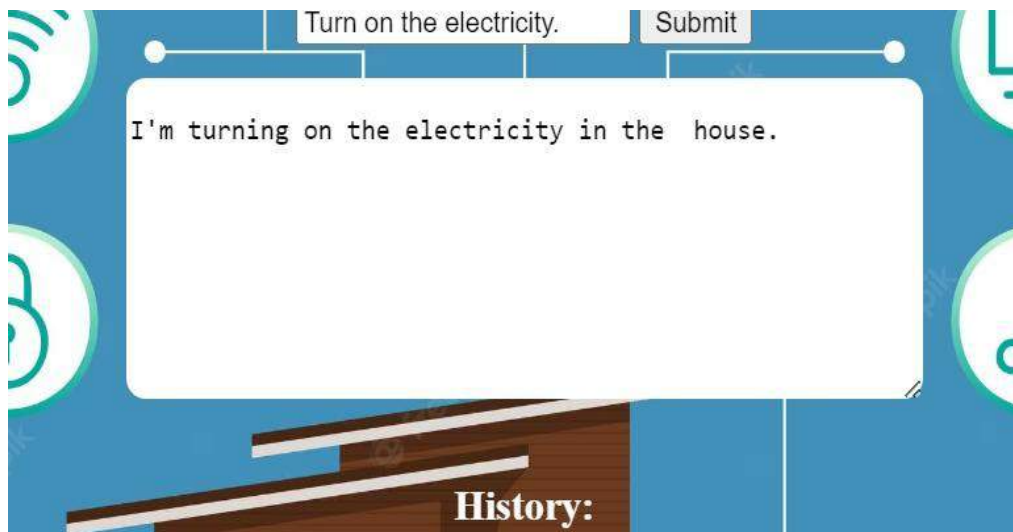
1. Deschiderea aplicației



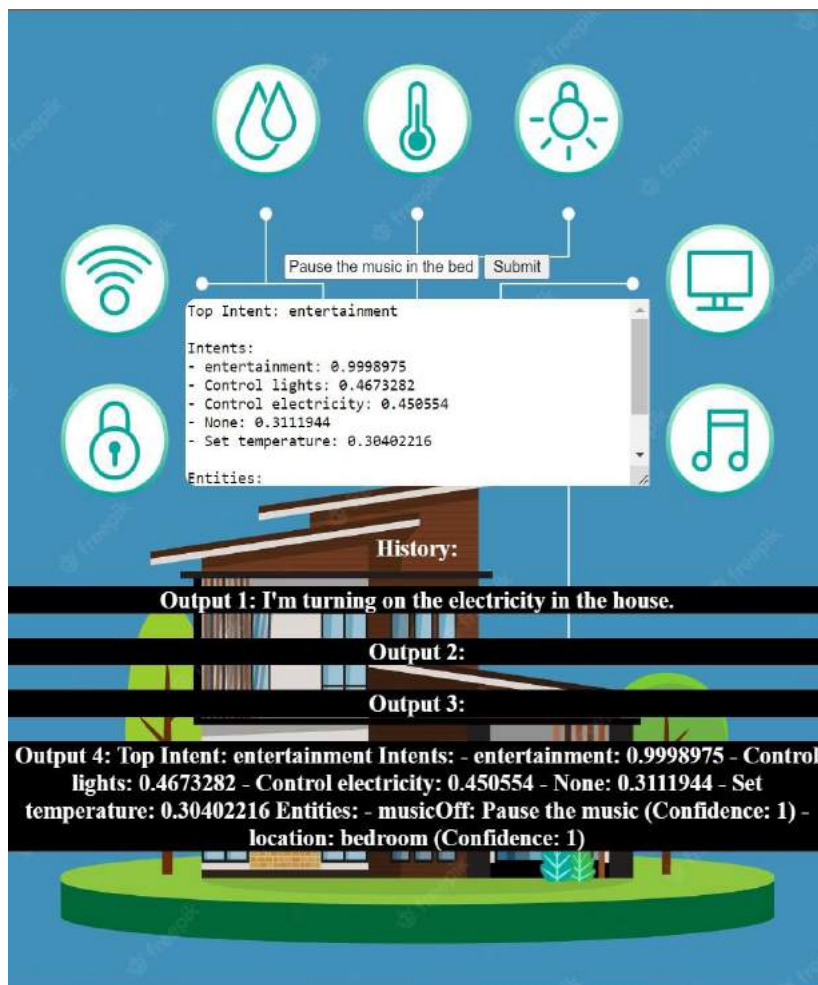
2. Introducerea textului în casuta destinată



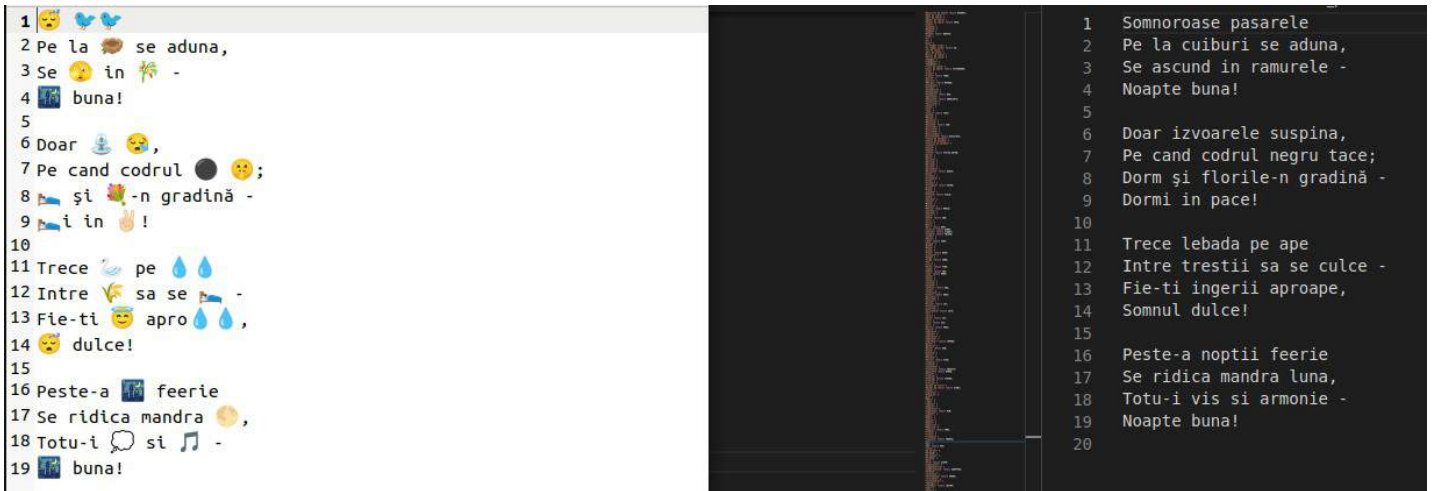
3. Primirea răspunsului în funcție de comanda dată de la API



4. Introducerea unui nou text si vizualizarea raspunsului, dar si a istoricului comenzilor introduse



Other Projects



Other Projects

```
Assistant: Hello! I am your assistant and today I am going to help you choose your clothes. :)
User: Hello!
Assistant: Based on what criteria would you like to dress yourself? The available criterias are: Event,
User: Event
Assistant: The event that you are attending is a wedding, an office meeting, a brunch date or a night out
User: Night out
Assistant: Would you like to wear a dress or a pair of pants?
User: Pants
Assistant: Is it hot or cold outside?
User: Cold
Assistant: You could wear:
    A white shirt, a pair of navy pants, a black blazer and a pair of black shoes.
    A white shirt, a pair of black pants, a green blazer and a pair of black shoes.
    Which option will you wear?
User: Second
Assistant: Ok, then you are ready for today. Enjoy!
```

Medical Leaflet Processor Transformers-HuggingFace

1 Introducere

Proiectul pe care l-am gândit este o aplicație numită "Drugs Leaflet Processor" care utilizează tehnologia PaddleOCR pentru a extrage text din imagini și oferă utilizatorilor un rezumat al informațiilor referitoare la medicamentele căutate. Aplicația permite utilizatorilor să facă o fotografie la ambalajul medicamentului și să obțină în avans informații despre utilizare și o evaluare generală a acestuia.

Proiectul "Drugs Leaflet Processor" are o legătură indirectă cu gramatica unei limbi în contextul materiei "Limbaje de formatare și translaoare". Deși nu este un aspect central al proiectului, gramatica unei limbi este implicată în procesul de recunoaștere a textului și extragerea informațiilor relevante din prospectele medicamentelor.

În cadrul aplicației, textul extras din imaginile cu pachetele de medicamente este prelucrat și analizat pentru a identifica informațiile specifice despre medicamentul căutat. Aceasta implică analiza sintactică și semantică a textului pentru a extrage entități cheie și a identifica structura și conținutul relevant.

De exemplu, în procesul de tokenizare, textul este divizat în unități mai mici, precum cuvinte sau simboluri, folosind reguli și modele lingvistice. Aceasta poate include separarea cuvintelor compuse sau identificarea prefixelor și sufixelor pentru a atribui sens corespunzător fiecărei unități.

În plus, atunci când se aplică procesul de sumarizare, gramatica limbii este esențială pentru a selecta frazele și informațiile relevante, păstrând în același timp coerența și înțelegerea generală a textului.

Deși proiectul se concentrează în primul rând pe procesarea și analiza prospectelor medicamentelor, înțelegerea gramaticii limbii și aplicarea regulilor lingvistice ajută la realizarea unui proces de extragere și sintetizare a informațiilor mai precis și mai eficient.

Astfel, proiectul "Drugs Leaflet Processor" ilustrează importanța și aplicabilitatea conceptelor de gramatică și prelucrare a limbajului natural în domeniul translaoarelor și limbajelor de formatare, evidențiind modul în care acestea pot fi utilizate pentru a extrage și a analiza informații din texte structurate sau semistructurate.

2 Structura proiectului

Aplicația "Drugs Leaflet Processor" este dezvoltată folosind Hugging Face, o platformă de dezvoltare și distribuție de modele de învățare automată. Platforma Hugging Face oferă o gamă largă de modele preantrenate, instrumente și resurse pentru a dezvolta aplicații bazate pe învățarea automată.

Aplicația noastră este disponibilă și în Google Colab, un mediu de programare bazat pe web care permite rularea și partajarea de cod Python. Puteti accesa și rula aplicația noastră în mediul Google Colab.

3 Analiza a implementării

Pentru implementarea acestei aplicații, am utilizat bibliotecile PaddleOCR, Transformers și Gradio. Am folosit PaddleOCR pentru a extrage textul din imagini, Transformers pentru a genera un rezumat al informațiilor despre medicament și Gradio pentru a crea interfața grafică a aplicației.

Pentru extragerea textului din imagini, am folosit clasa PaddleOCR și am specificat limba corespunzătoare în funcția inference.

Pentru generarea rezumatului informațiilor despre medicament, am utilizat modelul preantrenat google/pegasus-xsum și tokenizerul corespunzător în funcția get summary.

Datele despre medicamente sunt stocate în fișierul CSV Drugs.csv, iar funcția inference primește o imagine și limba dorită și returnează rezumatul informațiilor despre medicament.

3.1 Tokenizare

Pentru tokenizare, am utilizat pipeline-ul Transformers. Pipeline-ul este o abordare simplă și eficientă pentru utilizarea modelelor preantrenate, oferind o interfață ușor de utilizat pentru diferite sarcini, inclusiv tokenizare.

Pipeline-ul de tokenizare din Transformers vă permite să încărcați și să utilizați un tokenizer preantrenat cu doar câteva linii de cod. În funcția `get_tokens()`, am utilizat clasa `AutoTokenizer` pentru a încărca un tokenizer preantrenat pentru sarcina noastră specifică.

`AutoTokenizer` face parte din biblioteca Transformers și ne permite să încercăm un tokenizer potrivit pentru modelul dorit. Folosind `AutoTokenizer.from_pretrained()`, putem specifica numele sau calea către tokenizerul preantrenat pe care dorim să-l utilizăm. În cazul nostru, am folosit tokenizerul `d4data/biomedical-ner-all`.

După încărcarea tokenizerului, am folosit pipeline-ul NER pentru a procesa textul. Pipeline-ul NER se ocupă de recunoașterea entităților cu ajutorul modelului preantrenat și tokenizerului asociat. Rezultatul returnat este o listă de dicționare, fiecare dicționar conținând informații despre o entitate recunoscută în text.

```
1 def get_tokens(text):
2     ## auto tokenizer trained on biomedical data
3     ## we'll use this in our cool pipeline
4     tokenizer = AutoTokenizer.from_pretrained("d4data/biomedical-ner-all")
5     model = AutoModelForTokenClassification.from_pretrained("d4data/biomedical-ner-all")
6     # pass device=0 if using gpu
7     pipe = pipeline("ner", model=model, tokenizer=tokenizer, aggregation_strategy="
        simple", ignore_labels=[], grouped_entities=True, ignore_subwords=True)
8
9     ## transform to list with by creating an ordered dict from list
10    ## to get rid of duplcates *wink*
11    no_duplicates=OrderedDict.fromkeys(text)
12    ## then, transform the dictionary back to list
13    words = list(no_duplicates.keys())
14
15    print(words)
16
17    ## get rid of things like mg, drug, medication bcs
18    ## those get that score close to 90% and that s no good for our project
19    stop_words=['mg', 'drug', 'medication']
20    whitelisted_words = [word for word in words if word.lower() not in stop_words]
21
22    ## send that data to the pipe
23    pipe_result = pipe(whitelisted_words)
24    pipe_result = [item for sublist in pipe_result for item in sublist]
25
```



```

26  ## as an output we'll get a list of dictionaries, so it's time to filter
27  recognized_medication = []
28  for dict_entity in pipe_result:
29      if dict_entity['entity_group']=='Medication':
30          recognized_medication = dict_entity['word']
31          break
32
33  print('recognized medication=', recognized_medication)
34
35  return recognized_medication

```

3.2 Sumarizare

Pentru sumarizare, am utilizat clasa `AutoModelForSeq2SeqLM` pentru a încărca modelul preantrenat Pegasus-XSUM, care este specializat în sarcina de sumarizare a textului.

Clasa `AutoModelForSeq2SeqLM` face parte din biblioteca `Transformers` și ne permite să încărcăm un model preantrenat pentru sarcina de generare a secvențelor, cum ar fi sumarizarea. Am utilizat `AutoModelForSeq2SeqLM.from_pretrained()` pentru a încărca modelul `google/pegasus-xsum`.

Pentru a pregăti textul pentru generarea sumarului, am folosit clasa `AutoTokenizer` pentru a tokeniza textul de intrare. Tokenizarea se realizează cu ajutorul tokenizerului asociat modelului preantrenat. Apoi, am codificat tokenii utilizând metoda `encode()` a tokenizerului. Acest proces de encoding transformă textul într-o reprezentare numerică adecvată pentru modelul preantrenat.

După generarea sumarului utilizând modelul preantrenat, am utilizat metoda `decode()` a tokenizerului pentru a decoda secvența de tokeni într-un text uman-readable. Aceasta elimină tokenii speciali și transformă rezultatul într-un rezumat inteligibil.

Prin utilizarea clasei `AutoModelForSeq2SeqLM`, a tokenizerului și procesului de encoding și decoding, am putut realiza sumarizarea informațiilor despre medicament în cadrul aplicației noastre.

```

1  def get_summary(text_long):
2      ## Get the pretrained model from google
3      model = AutoModelForSeq2SeqLM.from_pretrained('google/pegasus-xsum')
4
5      ## Get the tokenizer from google
6      ## The tokenizer is responsible for splitting the text
7      ## into tokens and encoding them appropriately for the model.
8      tokenizer = AutoTokenizer.from_pretrained('google/pegasus-xsum')
9      tokens_input = tokenizer.encode("summarize: " + text_long, return_tensors='pt',
10                                     max_length=512, truncation=True)
11
12     ## The encoded tokens are then passed to the
13     ## Pegasus-XSUM model's generate function to generate the summary
14     ## The generate function performs the actual
15     ## summarization using the pretrained model
16     ids = model.generate(tokens_input, min_length=80, max_length=120)
17
18     ## The generated summary is obtained as a sequence of token IDs
19     ## These IDs are decoded into a human-readable summary using
20     summary = tokenizer.decode(ids[0], skip_special_tokens=True)
21     print(summary)
22     return summary

```

3.3 Inference

Subcapitolul "Inference" prezintă procesul de inferență al aplicației, în care se utilizează biblioteca PaddleOCR pentru a extrage textul dintr-o imagine și se obțin informații relevante despre medicamentul căutat.

Funcția `inference` primește două parametri:

- `img`: imaginea pentru care se efectuează inferența
- `lang`: limba utilizată pentru recunoașterea caracterelor în imagine

Funcția utilizează biblioteca PaddleOCR pentru a extrage textul din imaginea dată prin apelul `ocr.ocr(img, cls=False)[0]`. Rezultatul este o listă de linii de text identificate în imagine.

Textul relevant pentru căutarea medicamentului este extras din rezultat utilizând linia `txts = [line[1][0] for line in result]`.

Apoi, se utilizează funcția `get_tokens` pentru a tokeniza textul și a obține numele medicamentului căutat.

Numele medicamentului este pregătit pentru a fi căutat în fișierul CSV prin convertirea sa în majusculă utilizând metoda `capitalize()`.

Se încarcă informațiile despre medicamente din fișierul CSV folosind `pd.read_csv('Drugs.csv').drop_duplicates()`

Rândul corespunzător medicamentului căutat este identificat în fișierul CSV prin linia `filtered_row = leaflets[leaflets['Drug'] == medication_name].head(1)`.

Informațiile despre medicament sunt extrase din coloana "Information" utilizând `filtered_row['Information'].values.astype(str)[0]`.

4 Exemple

Pentru a utiliza aplicația, puteți încărca imagini de exemple și alege limba corespunzătoare din meniul derulant. Puteți utiliza exemplele de cod de mai jos:

- `levofloxacin.jpg` - Imagine cu ambalajul medicamentului Levofloxacin, în limba engleză.
- `azythromicin.jpg` - Imagine cu ambalajul medicamentului Azithromycin, în limba engleză.

5 Concluzie

Proiectul "Drugs Leaflet Processor" demonstrează utilizarea eficientă a unor tehnologii și biblioteci precum PaddleOCR, Transformers și Hugging Face pentru a dezvolta o aplicație practică de procesare a prospectelor medicamentelor. Aplicația oferă utilizatorilor posibilitatea de a obține informații relevante despre medicamente prin simpla încărcare a imaginilor cu pachetele de medicamente.

Prin integrarea funcționalităților de recunoaștere a textului, tokenizare, sumarizare și căutare în baza de date, aplicația noastră oferă o soluție rapidă și convenabilă pentru a obține informații utile despre medicamentele căutate.

Dezvoltarea acestui proiect ne-a permis să explorăm și să aplicăm concepte importante din domeniul prelucrării limbajului natural, precum și să ne familiarizăm cu instrumentele și tehnologiile cheie utilizate în dezvoltarea de aplicații bazate pe învățarea automată.

References

- PaddleOCR Demo Source Code:
<https://huggingface.co/spaces/phlippseitz/Image-Text-Extraction-PaddleOCR/blob/main/app.py#L20>
- Text Summarization with Transformers:
<https://rubikscore.net/2022/04/25/text-summarization-with-huggingface-transformers/>